

Advanced RAG

Hybrid Retrieval, Re-ranking and SQL RAG

Hybrid RAG • BM25 • Re-ranking • Text2SQL • MindSQL

1. Topic Overview

Session 5 built on the previous session's recap of chunking strategies and RAG types, then moved into hands on work. The session had two parts: a recap of the RAG landscape covered so far, followed by a live build of a hybrid RAG pipeline, a walkthrough of re-ranking, and an introduction to SQL RAG (text to SQL), including a tour of MindSQL, the open source Text2SQL library the instructor built and maintained. The cohort's second assignment, covering everything learned about RAG so far, was announced for release the next day.

Topic	What was covered
Recap of RAG types	Vector (naive) RAG, vector less (keyword) RAG, hybrid RAG, GraphRAG, SQL RAG, and page index (reasoning based) RAG, with their relative complexity, usefulness, and cost.
Hybrid RAG hands on	Building a pipeline over a CSV FAQ file, a PDF guide, and (conceptually) a database, using dense and sparse embeddings together in Qdrant.
Re-ranking	Why an initial retrieval ranking can be wrong, the bi-encoder and cross-encoder approaches, and when re-ranking is worth the added cost and latency.
SQL RAG	How text to SQL pipelines work, what an LLM needs to generate good SQL, a walkthrough of MindSQL, and a production pattern using a JSON query interface.

Industry Context

Text to SQL is one of the most common problems AI teams are asked to solve, and it comes up often in interviews. The instructor built an early career around this exact problem and later used a similar approach on petabyte scale production databases at a previous company.

2. Recap: The RAG Landscape So Far

Before the hands on portion, the session recapped the different types of RAG covered in earlier classes, since many learners asked for it. Vector RAG (naive RAG) selects the top K elements from a vector store using semantic similarity. Vector less RAG does not need embeddings at all: it uses algorithms such as BM25 or TFIDF, the kind of scoring an Elasticsearch or OpenSearch engine already provides, and works best for code, jargon, IDs, citations, and precise keyword queries, but is weak on semantic understanding. GraphRAG builds a context graph from the documents and uses graph traversal to answer a question; it is complex to implement but solves problems that other RAG types cannot. Page index, also called reasoning based RAG, builds a hierarchical tree index over a document the way a book's index works, and an LLM traverses that tree to find the right chunk; it is highly accurate but slow and resource intensive, so it is currently mostly used in legal and financial domains as an active area of research, not a default choice.

RAG type	Complexity to implement	Usefulness	Notes
Vector (naive) RAG	Low	Baseline	Top K semantic retrieval from a vector store.
Keyword (vector less) RAG	Low	Low on its own	Uses BM25 or TFIDF, no embeddings needed.
Hybrid RAG	Medium (needs engineering)	Highest, most production systems use this	Combines dense and sparse retrieval, then merges results.
GraphRAG	High	High for specific problems only it can solve	Also high cost; a recorded module with Neo4J is planned.
SQL RAG	Medium, depends on the database and use case	High, very common industry need	Generates and executes SQL rather than retrieving text chunks.
Page index (reasoning based) RAG	High	Very accurate but slow	Expensive and inefficient; still experimental, mostly legal and financial domains.

Key Insight

Hybrid RAG is the most used approach for real world problems because it runs a vector search and a keyword search in parallel, then re-ranks the combined results, so it captures both semantic meaning and keyword precision. HNSW is the algorithm used to retrieve points from a vector store in any RAG approach that relies on one.

3. Building a Hybrid RAG Pipeline: Hands On

The hands on exercise moved to a new problem: a telecom assistant that has to answer questions from three kinds of sources at once, an FAQ spreadsheet, a PDF guide, and (conceptually) a database, rather than the single HR policy document used in the previous session.

3.1 Ingesting Structured and Unstructured Sources

The FAQ file, a CSV with 25 question and answer pairs plus a category column, was loaded with Pandas and converted into LangChain Document objects, one document per row. This is a form of fixed size, layout aware chunking: because the source is a table, each row is a natural, self sufficient chunk, and the row's category, source, and row ID were carried along as metadata. The PDF guide, being unstructured, was chunked differently: it was converted with Docling into a Docling document, then split with a hybrid chunker that produces hierarchical chunks and further splits any chunk larger than 128 tokens. That produced 43 chunks from the PDF, versus 25 from the FAQ file, for 68 documents combined. Because the PDF is unstructured, only the source field could be added as metadata; the instructor noted a category field could still be added later using headings and subheadings as metadata, or by passing each chunk through a small classification model or a lightweight LLM call at ingestion time (a one time cost, since ingestion, unlike search, only happens once).

Key Insight

Chunking is not about naming the right technique. The instructor was direct about this: many chunking names are mostly vocabulary to sound precise. The real skill is looking at the actual shape of the document in front of you and designing (or inventing) the chunking approach that fits it, rather than reaching for a textbook label.

3.2 Dense and Sparse Embeddings Together

Hybrid RAG's core idea is to use two embeddings at once: a dense embedding model for semantic meaning (the session used all-MiniLM-L6-v2 via Hugging Face) and a sparse embedding model for keyword matching (Qdrant's own BM25 implementation, loaded through the FastEmbed library, which ships Qdrant's recommended dense and sparse models optimized to run efficiently on a local machine). Both embeddings were generated for all 68 chunks and stored together in a Qdrant collection, with the retrieval mode set to hybrid so that a search considers both embeddings and returns documents that either or both retrieval methods surface. The RAG chain itself used LangChain's `create_retrieval_chain` and `create_stuff_document_chain` utilities rather than a hand rolled retrieval function, since this is such a common pattern that LangChain already provides it, retrieving the top 5 documents per query.

Embedding type	Captures	Example query it handles well
Dense (semantic)	Meaning and context, even without matching keywords	"My phone shows full bars but I cannot load any website" (a connectivity issue, described without the word connectivity)
Sparse (keyword, BM25)	Exact terms, jargon, IDs, acronyms	"Where are APN settings?" (APN has little semantic meaning on its own)

Live queries against the pipeline showed both effects: a semantic question about connectivity retrieved the right troubleshooting chunk without using the word connectivity, and a keyword heavy question about APN settings still retrieved the correct chunk, because the sparse embedding matched on the term directly. The instructor was clear that this pipeline is not yet production ready: passing every retrieved chunk (both dense and sparse hits) straight to the LLM adds noise, tokens, and latency, which is exactly the problem re-ranking addresses next.

3.3 Adapting Hybrid RAG to an Existing Elasticsearch or OpenSearch Setup

A recurring, practical question was how to add hybrid RAG on top of a legacy system that already runs on Elasticsearch or OpenSearch rather than a vector database. The recommended approach: keep the existing keyword index as is, since Elasticsearch and OpenSearch both provide their own vector store, add semantic embeddings there too, run both a keyword and a semantic search, retrieve a broader pool (for example the top 10 to 20 from each), deduplicate any documents both methods returned (a document both approaches ranked highly is itself a strong relevance signal), and re-rank the combined pool down to the final top 5. Migrating everything into a brand new vector database is rarely worth the overhead; companies that already invested years into Elasticsearch, OpenSearch, or Postgres (with pgvector) generally extend what they have rather than re-index everything elsewhere.

4. Re-ranking

Re-ranking exists because the order documents come back from a vector search is not always the order of true relevance to the question. The common pattern is to over fetch (for example retrieve the top 10 or 15 documents when only 5 are needed) and then use a separate re-ranker model to reorder that larger pool before picking the final top few. The instructor compared this to asking a doctor for a second opinion before a decision: you do not rely on a single signal for something that matters.

4.1 Bi-encoder vs Cross-encoder

There are two common re-ranking methods. A bi-encoder generates a separate embedding for the query and a separate embedding for each document, using two instances of the same (or a different) embedding model, then ranks documents by a distance metric such as cosine similarity. A cross-encoder, the approach used in the session, concatenates the query and a single document into one input, passes that combined text through a transformer model in one pass, and produces a single relevance score for that pair, repeating this for every

candidate document and sorting by score afterward. The session used a Hugging Face cross-encoder, ms-marco-MiniLM-L6-v2 (a Microsoft model trained specifically for re-ranking, with roughly 84 million downloads), to take the top 10 documents from the hybrid retriever down to the top 3 most relevant.

Key Insight

A different model is deliberately used for re-ranking than for retrieval. If the same model, and the same bi-encoder style comparison, were used for both steps, re-ranking would add latency without adding any real signal, since it would just be repeating the same judgment. A cross-encoder cannot be swapped in for the retrieval step itself because it needs a document in hand to score it against a query; it cannot search a whole corpus on its own.

4.2 When Is Re-ranking Worth It

Re-ranking has real costs: extra latency (since it scores every candidate document against the query one at a time) and extra compute. Whether it is worth adding depends on evaluation results, a topic the session deferred to the upcoming evals class, but the instructor gave the underlying logic here. Build the RAG pipeline, evaluate it (for example on retrieval accuracy and recall), and if accuracy is already high (the example given was 98 percent) in a scenario that is not high stakes, extra latency from a re-ranker is probably not worth it. If accuracy is low (the example given was 70 percent), add the re-ranker. The decision should always be grounded in the actual problem, the trade offs available, and what the user experience actually needs, not applied as a default step.

Question	Answer from the session
Does re-ranking use the same embeddings as retrieval?	No. A different model is used on purpose, so the two steps are not just repeating the same signal.
Can re-ranking run once a day instead of per query?	No. Re-ranking has to happen every time retrieval happens, because it re-ranks the specific query plus its specific retrieved chunks. Semantic caching is a separate technique for cost optimization, to be covered later.
Does the cross-encoder generate embeddings?	No. It only outputs a relevance score for a query and document pair; the bi-encoder is the one that produces separate embeddings.
Is it common to re-rank keyword hits and semantic hits together?	Yes. A common pattern is to fetch a pool from both methods, deduplicate documents both surfaced, and re-rank the combined pool.

5. SQL RAG and MindSQL

5.1 The Core Idea: Text to SQL

SQL RAG solves a different kind of problem than document retrieval: the user asks a natural language question, an LLM generates a SQL query, that query runs against a real database, the result comes back to the model, and the model turns the result into a plain language answer. The instructor framed the mental model using an intern analogy: to get a new intern productive on a SQL codebase, you would show them the DDL (the schema, as create statements), give them business context about what the schema means, and give them examples of SQL already written for the domain. An LLM needs exactly the same three things: DDL schemas, business documentation, and example question and SQL pairs, and with that context it can write SQL the same way the intern could contribute code.

Ingredient	What it is	Typical chunk size
DDL schemas	The create statements for each table	One table's DDL per chunk
Example question and SQL pairs	Real questions users have asked, with the correct SQL for each	One question and SQL pair per chunk
Documentation	Business meaning behind columns, enums, currencies, and similar context	One paragraph per chunk

The live demo used LangChain's SQLDatabase utility to connect to a small tickets database (one table, with columns for ticket ID, category, issue type, description, resolution, and status), used LangChain's create_sql_query_chain to generate raw SQL from a natural language question, cleaned the LLM's markdown formatted output down to plain SQL, ran that SQL against the database, and passed the result back to the LLM to produce a natural language answer. Sample questions such as "how many tickets are in each category" and "what is the most common issue type in the connectivity category" both produced correct SQL and correct answers. The instructor was direct that this minimal chain is a mental model, not something to rely on for a real system, and pointed the cohort toward a more complete architecture next.

5.2 MindSQL: A Real Text2SQL Project

MindSQL is the open source Text2SQL library the instructor built and was the primary contributor to; it has been used by roughly 450 people and, according to the instructor, is used in production by teams in China, among others. It defines three core interfaces, for a database, a vector store, and an LLM, connected together into one pipeline. Ingestion happens through an index_all_ddls method that connects to the target database, fetches every DDL (create statement), and indexes each one into the vector store, alongside indexed example question and SQL pairs and documentation. At query time, the relevant DDL, examples, and documentation are retrieved (a re-ranker can be added here too), assembled into a prompt, and given to the LLM to generate SQL, which is then executed and turned into an answer. The instructor noted this same architecture worked

even with an older, much weaker model (Llama 7B), which underlines that the context you assemble, not the newest possible model, is often what determines whether a text to SQL system works.

Industry Context

MindSQL is no longer actively maintained by the instructor, since ownership of the repository sits with a former employer. The recommendation for the cohort: treat MindSQL as a reference architecture, and if building something similar today, use it as a starting pattern and let an AI coding assistant help extend it, rather than depending on the library itself for a production system.

5.3 A Production Pattern for Dangerous or Complex Databases

For databases that are too complex, too verbose (the example given was ClickHouse, an OLAP columnar database where queries commonly run to two pages of SQL), or too risky to expose directly to an LLM or an application, the instructor described a pattern used in a previous production system: build a query interface layer that sits between the application and the database. The application (or an LLM based application) sends a small JSON query to this interface; the interface is responsible for turning that JSON into the actual, possibly very long and dialect specific, SQL query, executing it, and returning a JSON response. This keeps the LLM working with a format it is very familiar with (JSON, roughly 200 tokens in the example given, versus 1000 to 1500 tokens for the equivalent raw SQL), makes the application database agnostic, and lets all authorization, validation, and query safety logic live in one place rather than being scattered across every caller.

5.4 SQL RAG vs Keyword RAG

A recurring point of confusion was whether SQL RAG and keyword RAG are the same idea, since both can feel keyword driven. They are not. Keyword RAG (BM25 or TFIDF) searches a document store for keyword matches and returns text chunks to the LLM. SQL RAG generates and executes a structured query against a database and returns a computed, structured result; there is no vector search of unstructured text involved in the core loop, though a vector store can still be used to retrieve the most relevant DDL, examples, and documentation to build the prompt.

6. Guardrails and Security for SQL RAG

Several questions focused on the real risk of connecting an LLM to a production database: an LLM could, in principle, be led into generating a destructive statement such as a DELETE or DROP. The session covered a few concrete mitigations: validate that generated SQL only contains a SELECT and rejects DML or DDL statements outright, using rule based checks such as regex rather than relying on the LLM alone to behave; use an intent classifier to catch and reject malicious sounding requests before SQL generation even happens; and, for the highest risk systems, avoid using an open source Text2SQL library directly against a production connection and instead build the query and validation logic from scratch. The instructor was direct on this point: if security is the concern, do not connect an off the shelf library straight to a production database.

7. Engineering Mindset Notes

A theme repeated throughout the session: chunking strategies, RAG types, and re-ranking methods are reference points for your thinking, not a checklist to follow mechanically. The instructor pushed the cohort to ask, for any new problem, what the data actually looks like and what a good engineer would build to solve that specific case, rather than reaching for the nearest named technique. On environment setup frustration (some learners wanted to move off Google Colab immediately), the instructor asked for patience: the plan is to spend the current weeks on RAG and agent concepts in a shared environment, then move to local, and eventually AWS, deployment once the underlying concepts are solid, specifically to avoid learners getting lost in environment setup before they have understood the core ideas.

8. Session Output and Next Steps

Item	Detail
Assignment	The cohort's second assignment, covering all RAG concepts learned so far, was to be released the next day.
Suggested prep	Revisit the past three live sessions before starting the assignment.
Homework style exercise	Take the top 3 re-ranked documents from the hands on notebook and complete the pipeline by passing them to the LLM to generate a final answer (this last step was demonstrated conceptually but not run live).
What is next	The next live session moves into agentic AI, starting with AI agents. Evaluations (evals), including how to judge retrieval accuracy and decide when a re-ranker is worth it, and topics such as self-RAG and corrective RAG, are planned for later once agent concepts (the ReAct loop, self-correction, context management) are in place.
Recorded bootcamp modules	GraphRAG, using Neo4J, is planned as a recorded module rather than a live session; a Pandas primer already exists in the recorded bootcamp for anyone who needs it.

9. Quick Revision Summary

Term	What it means
Hybrid RAG	Retrieval that combines a dense (semantic) embedding and a sparse (keyword, BM25) embedding, run in parallel and merged, used by most production systems.
FastEmbed	Qdrant's own library and set of dense and sparse embedding models, optimized to run efficiently on a local device.
Dense embedding	A semantic, contextual vector representation of text.
Sparse embedding	A keyword based representation (for example BM25), with mostly zero values except at matching terms.
Re-ranking	Taking a broader pool of retrieved documents and re-scoring them with a separate model to select the truly most relevant subset.
Bi-encoder	A re-ranking approach that embeds the query and each document separately, then compares them with a distance metric such as cosine similarity.
Cross-encoder	A re-ranking approach that concatenates the query and one document and produces a single relevance score for that pair in one pass.
SQL RAG (Text2SQL)	Generating and executing SQL from a natural language question, using DDL, documentation, and example question and SQL pairs as context.
MindSQL	An open source Text2SQL library, built by the instructor, that wires together a database, vector store, and LLM through a simple interface.
Query interface pattern	A JSON based middle layer between an application and a complex or sensitive database, used to keep SQL generation, validation, and authorization in one controlled place.
HNSW	The algorithm used to retrieve nearest neighbor points from a vector store during retrieval.